Swiss Joint Master in Computer Science

13081/43081 Systems Verification

Automated Verification

Prof. Ulrich Ultes-Nitsche

Department of Informatics, University of Fribourg

How automatic is automated verification?

- Well, fully!
- But it comes at a price ...
  - Up to now, the models we considered contained unbounded data types (e.g. `nat`).
  - They were therefore *infinite state*.
- To verify automatically, we require finite-state models!
  - Then we do not need induction anymore.
  - We can explore the *state space* of the model completely.

What are we going to do?

- modelling of systems in Promela (PROcess MEta LAnguage)
- verification of system models using Spin (Simple Promela INterpreter)
- representation of system properties in temporal logic
- understanding the verification algorithms

About Spin

- Gerard J. Holzmann. *The Spin Model Checker — Primer and Reference Manual*. Addison-Wesley, Pearson Education, September 2003, ISBN 0-321-22862-6.
- on-line manual at `http://spinroot.com/spin/Man/Manual.html`
- download Spin at `http://spinroot.com`

Modelling systems

- To analyse and verify systems, we first model them.
  - We do not verify the systems themselves but models thereof.
  - A model must describe all aspects of the system in such detail that it can be verified in a meaningful way.
- We introduce a language for modelling systems together with a software tool for simulating, analysing, and verifying models written in that language.

Running example: mutual exclusion

MutEx aims at preventing two processes from visiting a *critical section* simultaneously.

Model 1: MutEx

```
int my_turn = 0;
```

```
process 1
{
if (my_turn == 0){
      my_turn = 1;
      critical_section;
      my_turn = 0;
      }
}
```

```
process 2
{
if (my_turn == 0){
      my_turn = 1;
      critical_section;
      my_turn = 0;
      }
}
```

Does this solve MutEx?

The computer can help us

- We model systems in a language that a computer can parse and "execute".
- We ask a suitable program whether or not a model satisfies a given set of properties.
- It is one of the goals of this course unit to teach you how to do that.

How do we model systems?

- We use Promela.
- Promela is the input language to a program called Spin.
- Subsequently, we look at some basic Promela concepts.
  - In parallel, we model some simple example systems.
  - In parallel, we learn as well how to use Spin to analyse Promela models.

Some Promela data types

bit	$0, 1$
bool	*true*, *false*
byte	$0 \ldots 255$
short	$-2^{15} \ldots 2^{15} - 1$
int	$-2^{31} \ldots 2^{31} - 1$

Variable declarations in Promela (exemplarily)

```
bool test_result;

byte result;

byte result_2 = 12;
```

Promela's "operational objects": *processes*

- A Promela model consists of one or more processes:

```
proctype name (parameters) {
... process declaration ... }
```

- It contains (most of the time) a process **init**:

```
init { ... process declaration ... }
```

  - **init** is the process running initially. If **init** is missing, Spin will produce an error message.
  - exception: no error message when other processes are declared as **active**:

```
active proctype name (parameters) {
... process declaration ... }
```

Decisions: **if** statements

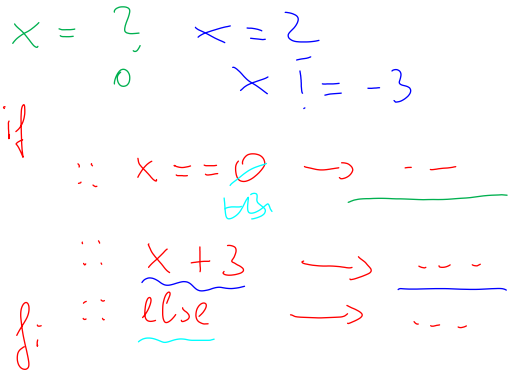
For the moment, we define if statements as follows:

```
if
      :: condition_1 -> statements_1
      :: condition_2 -> statements_2

      ...

      :: condition_n-1 -> statements_n-1
      :: else -> statements_n
fi
```

If **condition_i** is satisfied, **statements_i** **can be** executed $(1 \leq i < n)$.

statements_n are executed in the "else" case, i.e. **else** evaluates to *true* when **condition_1**, ..., **condition_n-1** **all** evaluate to *false*.

Conditions (e.g. in **if** statements)



- comparison of values:

equality:	==
inequality:	!=
less than:	<
greater than:	>
at most:	<=
at least:	>=

- logical expressions:

and:	&&
or:	||
not:	!

- all other expressions (incl. integer-valued ones)
  - value 0 is interpreted as *false*
  - all other values are interpreted as *true*

Assignments

we can assign values to variables (according to their type):

```
int num;
byte bb;

num = 42; /* assigns 42 */
bb = 932; /* assigns (932 mod 256) */
```

Starting processes

processes are started by:

```
run name ( parameters );
```

If a process declaration does not use parameters, we still must write empty brackets.

Parallel processes

How can we treat the parallel execution of several processes?

- If two or more process steps could be executed concurrently, we assume that one executes first.
  - interleaving semantics
- As we do not know which process will be first, we (or the verification system) select one of the concurrently executable process steps nondeterministically and execute it.

Parallel processes — an example

Let's see how processes can be started within **init**:

```
init { run process1(); run process2()};
```

- first, **init** ist running
- then **init** starts process1
- so, process1 is running in parallel with **init**
- therefore, either **init** starts process2 or process1 executes its first step
- ...

Usually we want all processes to start before analysing their behaviour.

- To avoid that process steps are concurrently executable, one uses the **atomic** ... statement.
- Whatever appears in **atomic** ... is considered as one big atomic execution step.

```
init { atomic { run proc1(); run proc2() }}
```

Example process declaration: the erroneous MutEx model

```
/* erroneous solution of mutex */
byte my_turn = 0;

proctype process1(){
    if
    ::my_turn==0 -> my_turn = 1;
            /* critical section */
            my_turn = 0;
    fi
}

proctype process2(){
    if
    ::my_turn==0 -> my_turn = 1;
            /* critical section */
            my_turn = 0;
    fi
}

init { atomic { run process1(); run process2() } }
```

MutExErr0.pml

Similar process declaration: the erroneous MutEx model

```
/* erroneous solution of mutex */
byte my_turn = 0;

active proctype process1(){
    if
    ::my_turn==0 -> my_turn = 1;
            /* critical section */
            my_turn = 0;
    fi
}

active proctype process2(){
    if
    ::my_turn==0 -> my_turn = 1;
            /* critical section */
            my_turn = 0;
    fi
}
```

MutExErr1.pml

The MutEx violation can be made visible

- Promela contains so-called **assertions**:

```
assert ( logical expression );
```

  - these are logical expressions that must evaluate to *true* whenever they are visited
    - if they evaluate to *false*, a simulation (or verification) run by Spin will stop with an error message

We can put assertions wherever we like in a Promela model.

Process positions

- In a process, e.g. “exampleProc”, we can put a label before some statement
  - a label is a name followed by a colon (“:”)
  - e.g. “critical:”
- then

```
exampleProc@critical
```

  evaluates to *true* if and only if “exampleProc” is at label “critical:”

A verifiable process declaration

```
/* erroneous solution of mutex */
byte my_turn = 0;
```

```
active proctype process1() {
if
   :: my_turn==0 -> my_turn=1;
      critical: /* critical section */
      my_turn = 0;
fi
}
```

```
active proctype process2() {
if
   :: my_turn==0 -> my_turn=1;
      assert (!(process1@critical)); /* critical section */
      my_turn = 0;
fi
}
```

MutExErr2.pml

Verification = "complete simulation"

Spin allows us to simulate all execution sequences of the model

- that assumes a finite-state representation of the model
- one calls this a complete simulation, or: verification
- if the model contains a bug, i.e. e.g. an assertion can be violated, then this violation will be found during the verification
- verification will terminate when the first error is found or when all executions of the model are explored

Verification of invariants

- Assertions are quite handy in verification, as we saw in the previous example — as long as we know where to place them.
- Invariants are properties of a system that must always hold.
  - If we used assertions to verify them, we would have to put the same assertion everywhere in our model. That's not so handy, is it?
- We can do better by using a special process in Promela, called a *never claim*.

never claims: processes that should never terminate successfully

- In Promela, we can define processes that must be unsuccessful.
  - For the moment, *unsuccessful* means that a process does not reach its end.
  - One such process per model.
- They describe what should never happen and are called *never claims*.
- Their declaration is absolutely simple:

```
never { statements };
```

- Process **never** must remain executable.
  - Once blocked, it remains blocked forever.
- When it can reach its end, a never claim is *successful* and indicates an erroneous situation.

A *never claim*'s semantics

We must extend our semantical model a tiny bit to define the execution of never claims:

- The never claim executes first, before any execution step of the remaining process model.
- The never claim and the remaining process model take execution steps alternatingly (in a ping-pong fashion)
- Whenever by adhering to that order of execution steps, the never claim can reach its end and terminates, verification fails (i.e., an error is detected) and the verifier will produce a counterexample (i.e., an execution path into the erroneous situation).

do-loops

```
do
    :: Statements
       ...
    :: Statements
od
```

do-loops (with a selection mechanism as in **if**-statements) are repeated until a **break** statement is executed or until one jumps out of the loop with an explicit **goto**

MutEx with a never claim

```
/* erroneous solution of mutex */
byte my_turn = 0;
```

```
active proctype process1(){
if
   :: my_turn==0 -> my_turn=1;
      critical: my_turn = 0;
fi
}
```

```
active proctype process2(){
if
   :: my_turn==0 -> my_turn=1;
      critical: my_turn = 0;
fi
}
```

```
never{
do
   :: process1@critical && process2@critical -> break;
   :: else;
od
}
```

MutExErr3.pml

Process IDs

-
```
run process()
```
instantiates a process and creates an ID of type **byte** to it
- this process ID can be assigned to a variable:
```
byte p;
p = run process();
```
- such a process instance can be accessed by:
```
process[p]...
```

The best nonworking MutEx model ever

```
/* erroneous solution of mutex */
byte my_turn, p1, p2;
```

```
proctype process(){
if
    :: my_turn==0 -> my_turn=1;
       critical: my_turn = 0;
fi
}
```

```
never{
skip;
do
    :: process[p1]@critical && process[p2]@critical -> break;
    :: else;
od
}
```

```
init { atomic { p1 = run process(); p2 = run process() } }
```

MutExErr4.pml

Another use of labels: **goto**

in process declarations, we can put labels at various positions

using the instruction

```
goto label_name
```

we can jump in a process to the position where the label "label_name" is found

Declaring constants

constants that can be used freely:

```
#define name value
```

symbolic constants as data type **mtype** of variables:

```
mtype = {name_1, name_2, name_3, ..., name_n};
mtype var;
```

(max 256 names)

mtype has the advantage that Spin uses these names in simulation output rather than just numbers

Let's move to a working MutEx solution for a change

We are going to consider Dekker's algorithm.

Dekker's MutEx algorithm

```
#define Aturn false
#define Bturn true

bool x, y, t;
```

```
active proctype A() {
    x = true;
    t = Bturn;
    if
        :: (y == false || t == Aturn)
           /* critical section */
    fi
    x = false
}
```

```
active proctype B() {
    y = true;
    t = Aturn;
    if
        :: (x == false || t == Bturn)
           /* critical section */
    fi
    y = false
}
```

MutExCorr1.pml

Dekker's MutEx algorithm with verification conditions

```
#define Aturn false
#define Bturn true

bool x, y, t;
```

```
active proctype A() {
    x = true;
    t = Bturn;
    if
        :: (y == false || t == Aturn)
           -> criticalA:
    fi
    x = false
}
```

```
active proctype B() {
    y = true;
    t = Aturn;
    if
        :: (x == false || t == Bturn)
           -> criticalB:
    fi
    y = false
}
```

```
never{
    do
        :: A@criticalA && B@criticalB -> break;
        :: else;
    od
}
```

MutExCorr2.pml

The underlying execution model

a statement is either *blocked* or *executable*

- assignments are always executable
- logical and arithmetic expressions are statements that are executable if and only if they do not evaluate to 0
  - $2 < 3$ or $42 >= 42$ are executable (evaluating to 1)
  - 42 is executable (evaluating to 42)
  - $x <= 26$ is executable if and only if the value of x is at most 26
  - $2 * (x + 3)$ is executable if and only if the value of x differs from -3
  - $42/21 - 2$ is not executable
- blocked statements are put on hold until they become executable

Execution model:
the truth about **if** statements (true for **do** loops, too)

a condition in an **if** clause is just a statement

- if it is executable, the **if** branch can be executed

```
if
    :: statement_1 -> more_statements_1
    :: statement_2 -> more_statements_2
       ...
    :: statement_n -> more_statements_n
fi
```

- if none of the first statements is executable, the **if** clause is not executable
- if many of the first statements are executable, one of the executable **if** branches is selected non-deterministically and executed
- "− >" is therefore synonymous to ";"
  - it just tells us that the author of the Promela models believes that what comes before should be interpreted as a condition
- **if** clauses with just one **if** branch can be omitted

Nonworking MutEx without **if**

```
/* erroneous solution of mutex */
byte my_turn = 0;
```

```
proctype process(){
    my_turn==0;
    my_turn=1;
    critical:
    my_turn = 0;
}
```

```
never{
skip;
do
    :: process[p1]@critical && process[p2]@critical; break;
    :: else;
od
}
```

```
init { atomic { p1 = run process(); p2 = run process() } }
```

MutExErr5.pml

Dekker's MutEx algorithm without **if**

```
#define Aturn false
#define Bturn true

bool x, y, t;
```

```
proctype A() {
    x = true;
    t = Bturn;
    (y == false || t == Aturn);
    criticalA: x = false
}
```

```
proctype B() {
    y = true;
    t = Aturn;
    (x == false || t == Bturn);
    criticalB: y = false
}
```

```
never{
    do
        :: A@criticalA && B@criticalB; break;
        :: else;
    od
}
```

```
init { atomic {run A(); run B()} }
```

MutExCorr3.pml

Does it work with arbitrary repetitions?

```
#define Aturn false
#define Bturn true

bool x, y, t;
```

active

```
proctype A() {
   do
      :: x = true;
         t = Bturn;
         (y == false || t == Aturn);
         criticalA: x = false
   od
}
```

active

```
proctype B() {
   do
      :: y = true;
         t = Aturn;
         (x == false || t == Bturn);
         criticalB: y = false
   od
}
```

```
never{
   do
      :: A@criticalA && B@criticalB; break;
      :: else;
   od
}
```

```
init { atomic {run A(); run B()} }
```

MutExCorr4.pml

Is our model fair?

Can we assure that a process that wants to access the critical section can indeed access it eventually?

Or can it happen that one process always gets priority over the other process?

We make progress: **progress** labels

in a Promela model, we can put at arbitrary positions a label

```
progress:
```

if the model is non-terminating, then each infinite run through the model must visit infinitely often a label

```
progress:
```

otherwise, Spin will produce an error message together with a counter example

Is our model fair?

```
#define Aturn false
#define Bturn true

bool x, y, t;
```

```
active proctype A() {
   do
      :: x = true;
         t = Bturn;
         (y == false || t == Aturn);
         progress: x = false
   od
}
```

```
active proctype B() {
   do
      :: y = true;
         t = Aturn;
         (x == false || t == Bturn);
         progress: y = false
   od
}
```

With this model, we cannot answer the question.

Is our model fair?

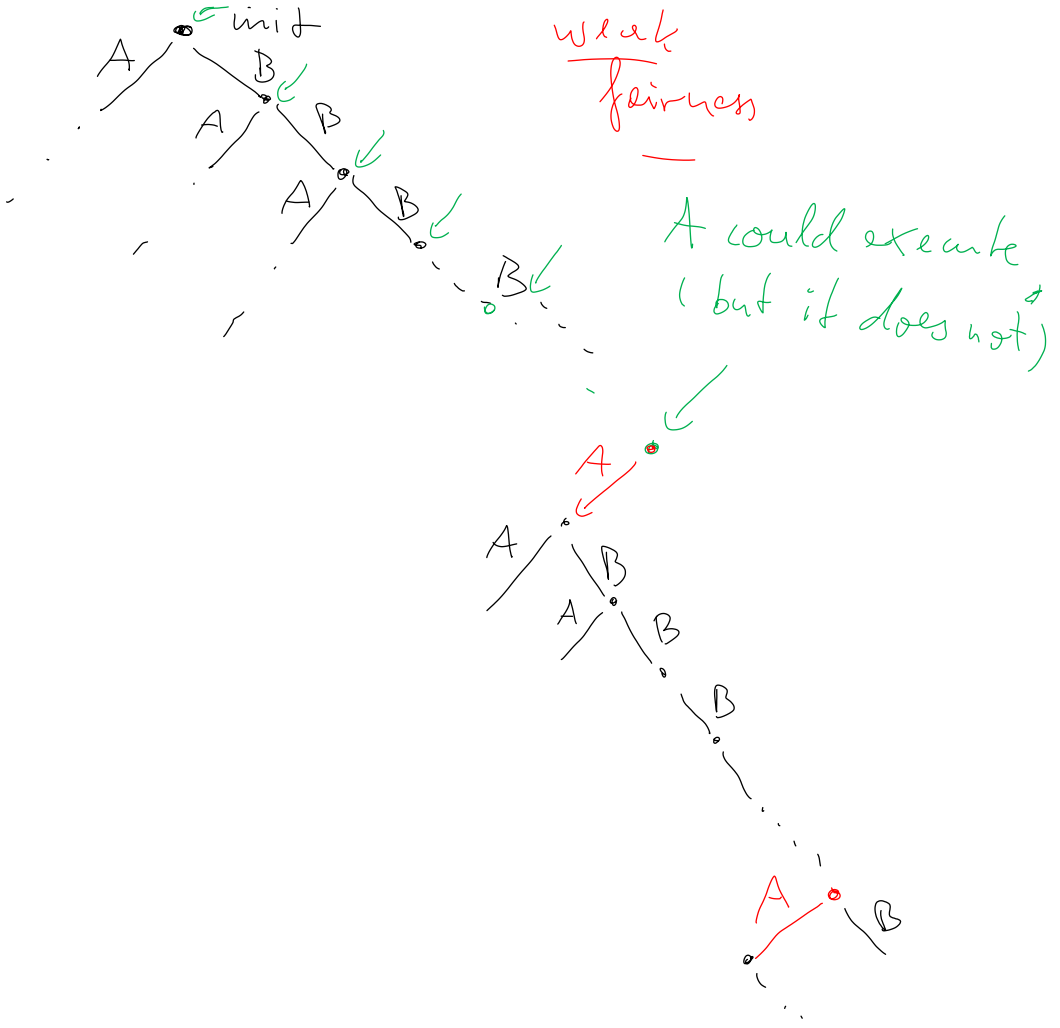
```
#define Aturn false
#define Bturn true

bool x, y, t;
```

```
active proctype A() {
   do
      :: x = true;
         t = Bturn;
         (y == false || t == Aturn);
         progress: x = false
   od
}
```

```
active proctype B() {
   do
      :: y = true;
         t = Aturn;
         (x == false || t == Bturn);
         y = false
   od
}
```

With this model, we can answer the question for process A.

MutExCorr5.pml


init
A
B
A
B
A
B
B
weak fairness
A could execute
(but it does not)
A
A
B
A
B
B
A
B

Is our model fair?

- it seems that process A can "starve"
- but only if it "voluntarily" makes no progress
  - that can be ignored by verifying the model with *weak fairness*
  - a run of the model is *weakly fair* if and only if each process that can make progress will eventually make progress

if a process is continuously enabled, then it will be executed eventually

Is our model fair?

```
#define Aturn false
#define Bturn true

bool x, y, t;
```

```
active proctype A() {
  do
    :: x = true;
       t = Bturn;
       (y == false || t == Aturn);
       progress: x = false
  od
}
```

```
active proctype B() {
  do
    :: y = true;
       t = Aturn;
       (x == false || t == Bturn);
       y = false
  od
}
```

With this model, we can answer the question positively (under weak fairness) for process A. (Symmetry gives us the same result for process B.)

MutExCorr5.pml

Is our model fair?

Is there a way to prove fairness not only for one process but for the two processes?

```
#define Aturn false
#define Bturn true

bool x, y, t;
```

```
active proctype A() {
   do
      :: x = true;
         t = Bturn;
         (y == false || t == Aturn);
         criticalA: x = false
   od
}
```

```
active proctype B() {
   do
      :: y = true;
         t = Aturn;
         (x == false || t == Bturn);
         criticalB: y = false
   od
}
```

```
never{ ... }
```

never claims: beyond never terminating successfully

- In Promela, never claims are processes that must be unsuccessful.
  - a never claim is *successful* when it can reach its end.
- But there's another way for a never claim to be successful:
  - we can use labels **accept:** in never claims
  - a never claim is also successful, when it can visit an **accept:** label infinitely often

Is our model fair?

What does a suitable never claim have to look like?

```
#define Aturn false
#define Bturn true

bool x, y, t;
```

```
active proctype A() {
  do
    :: x = true;
       t = Bturn;
       (y == false || t == Aturn);
       criticalA: x = false
  od
}
```

```
active proctype B() {
  do
    :: y = true;
       t = Aturn;
       (x == false || t == Bturn);
       criticalB: y = false
  od
}
```

```
never{ ... }
```

MutExCorr6.pml MutExCorr7.pml

"negative version":

"A critical" or "B critical" occur only finitely often.

↳ never claim

```
do
:: do
   :: accept: !(A critical A);
   od
:: skip;
od
```

critical A

critical A

critical B

critical A

critical A

critical A

critical B

each infinitely long execution through our model should visit "A critical A" and "B critical B" infinitely often.

(property the model should satisfy)

(the "opposite" of what will make the never claim successful)

! last occurrence

--- "A critical A" - - - - - does not contain "A crit".

To finish MutEx off: two more Promela constructs

- arrays:
  - ```
    data_type name[size];
    ```
  - example:
    - ```
      byte a[3];
      ```

      declares three byte variables: a[0], a[1], and a[2]
- process parameters:
  - ```
    proctype name (parameters) { ... statements ... }
    ```

    started by

    ```
    run name ( parameters )
    ```
  - example:
    - ```
      proctype test (byte b) { x = b }
      run test (42);
      ```

The most compact version or Dekker's algorithm

```
bool t;
byte p1, p2;
bool x[2];
```

```
proctype process(bool me, other) {
do
    :: x[me] = true;
       t = other;
       (x[other] == false || t == me);
       critical: x [me] = false;
od
}
```

```
never{
do
    :: do
          :: accept: !process[p1]@critical;
       od
    :: do
          :: accept: !process[p2]@critical;
       od
    :: process[p1]@critical && process[p2]@critical; break;
    :: skip;
od
}
```

```
init { atomic {p1 = run process(0,1); p2 = run process(1,0)} }
```

MutExCorr8.pml

That's all about MutEx

With this soft and gentle introduction to verification (of the MutEx property), I am going to stop this section.

Do you have questions?